

Validity Analysis: HUMANEVAL

Target context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Overall risk: **HIGH**

Deployment Context

We are a US enterprise software company evaluating whether an LLM can generate full-stack web application scaffolding from high-level requirements documents. Given a requirements spec, the system should produce database schemas, API route definitions, frontend component stubs, and deployment configurations. Our users are American software architects. We need to evaluate the LLM's code generation capabilities for this multi-layer task.

Dimension Scores

Dimension	Score	Rating	Confidence
Input Ontology ■	1	Serious concern	high
Input Content	2	Significant gaps	high
Input Form	1	Serious concern	high
Output Ontology ■	1	Serious concern	high
Output Content ■	1	Serious concern	high
Output Form ✓	2	Significant gaps	high
Average	1.3		

■ = highest concern ✓ = strongest dimension

Dimension Key

Abbr.	Dimension	Definition
IO	Input Ontology	Whether the benchmark's test case categories cover the query types expected in deployment.
IC	Input Content	Whether individual datapoint content is culturally and contextually appropriate for the target region.
IF	Input Form	Whether the input signal encoding (text, audio, image parameters) matches deployment conditions.
OO	Output Ontology	Whether the benchmark's output categories and scoring criteria reflect regionally valid decision boundaries.
OC	Output Content	Whether ground-truth labels align with the judgments of regional stakeholders.
OF	Output Form	Whether the expected output modality matches regional deployment needs and accessibility.

Overall Summary

HumanEval is a foundational benchmark for isolated Python function synthesis evaluated by unit-test pass/fail. For the deployment use case — multi-page PRD-to-multi-file-scaffolding generation graded by an internal architecture rubric — the benchmark is fundamentally misaligned across all four user-flagged HIGH-priority dimensions (IO, IC, IF, OO, OC) and substantially misaligned on OF. Five of six dimensions score 1–2. The only points of genuine contact are: (a) US English natural-language register, (b) typed Python function signatures, (c) text-in/code-out modality, and (d) a usable sandboxed-execution infrastructure that can serve as a lower-bound check for Python utility sub-components inside larger scaffolding outputs. The paper itself acknowledges most of the relevant limitations — long-specification handling [Q88], absence of qualitative architectural metrics [Q86, Q149], lack of automatic docstring evaluation [Q78], and distribution

mismatch with broader GitHub code [Q48]. Web research confirms dedicated benchmarks (DevBench, SWE-bench++, laC-Eval, DPlac-Eval, Multi-laC-Eval, FrontendBench, LongCodeBench, Prometheus 2) now exist that cover most of the validity gaps individually, though none combines all four artifact layers required here.

ID	Evidence
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)
Q78	"However, there is no similar way to evaluate docstring samples automatically. Therefore, we grade sample docstrings by hand, considering a docstring correct if it uniquely and accurately specifies the code body." (p.9)
Q86	"While evaluating code generation is well-studied (Xu et al., 2021; Helmuth & Spector, 2015; Pantridge et al., 2017), many existing metrics measure performance in tightly specified, constrained problem instances (e.g., string manipulation in FlashFill (Gulwani, 2011)). Therefore, we developed a set of qualitative metrics for measuring the capabilities of code generating models while controlling for the complexity and abstraction level of the specifications (Appendix D)." (p.10)
Q88	"Moreover, Codex struggles to parse through increasingly long and higher-level or system-level specifications." (p.10)
Q149	"Below we provide brief descriptions of such attributes and qualitative metrics, which are to be further discussed in a forthcoming paper along with associated results for Codex models." (p.25)

Practical Guidance

What This Benchmark Measures

HumanEval measures a model's ability to synthesize a single self-contained Python function from a short docstring and have it pass a small set of unit tests on algorithmic/mathematical problems. For the deployment, this corresponds to at most one narrow sub-component of the broader scaffolding task: isolated Python utility functions that might appear inside FastAPI route handlers or SQLAlchemy model methods. The strongest dimension is Output Form (modality match) and the weakest are Input Ontology, Output Ontology, and Output Content (each scoring 1).

Construct Depth

The benchmark probes functional correctness at a behavioral-contract level only. It does not probe: (a) framework-idiom adherence (FastAPI dependency injection, SQLAlchemy declarative models, Pydantic v2 validation, Spring Boot annotations), (b) cross-file/cross-language coherence, (c) architectural-convention adherence against an internal style guide, (d) long-context document parsing, or (e) infrastructure-as-code generation. The paper explicitly defers richer specification attributes (variable interdependencies, temporal reasoning, concurrency) to future work [Q149]. Quantitative ceilings reported in the literature for the layers HumanEval omits are sobering: <20% pass@1 on compositional laC [WEB-4], ~50% on simplest laC scenarios with 42.7% deployment-failure rate among syntactically correct templates [WEB-5], and 29%→3% degradation as coding-task context length grows [WEB-3].

What Else You Need

Comprehensive supplementation is required: (1) DevBench [WEB-12] or SWE-bench++ [WEB-9] for multi-stage / multi-language repository-level coverage; (2) laC-Eval [WEB-4] + DPlac-Eval [WEB-5] + Multi-laC-Eval [WEB-6] for the Dockerfile/Kubernetes/Terraform deployment-config layer; (3) FrontendBench [WEB-13] for React/Angular component-stub evaluation; (4) LongCodeBench [WEB-3] or LoCoBench [WEB-18] for long-context input form; (5) Prometheus 2 [WEB-14] as a rubric-judge framework with the org's architecture guide encoded as structured criteria, calibrated against a small human-annotated gold set per TRACE [WEB-16]; (6) a custom in-house harness for multi-file entity-name coherence (schema ↔ API model ↔ frontend prop types) since no public benchmark explicitly scores this. Framework-specific evaluation for FastAPI/SQLAlchemy/Pydantic/Spring Boot remains a confirmed gap (search gap 4 NOT FOUND) and requires custom benchmark construction using org-internal PRDs and the architecture guide as ground truth.

ID	Evidence
Q149	"Below we provide brief descriptions of such attributes and qualitative metrics, which are to be further discussed in a forthcoming paper along with associated results for Codex models." (p.25)

WEB-3	LongCodeBench documents Claude 3.5 Sonnet dropping from 29% to 3% as context length increases on coding tasks (https://arxiv.org/pdf/2505.07897)
WEB-4	IaC-Eval provides Terraform/HCL output-form evaluation HumanEval lacks (https://medium.com/gft-engineering/evaluating-llms-for-infrastructure-as-code-9f8b9ac4ca33)
WEB-5	DPIaC-Eval finds 42.7% of syntactically correct templates fail deployment, demonstrating ontology layers HumanEval cannot capture (https://arxiv.org/html/2506.05623v1)
WEB-6	Multi-IaC-Eval covers CloudFormation, Terraform, and CDK output forms (https://arxiv.org/pdf/2509.05303)
WEB-9	https://arxiv.org/pdf/2512.17419
WEB-12	DevBench covers full SDLC stages — illustrates richer output ontology the deployment requires (https://arxiv.org/abs/2403.08604)
WEB-13	FrontendBench covers front-end interactivity output forms (https://arxiv.org/html/2506.13832v2)
WEB-14	Prometheus 2 supports user-defined rubric criteria in pointwise and pairwise formats (https://arxiv.org/abs/2405.01535)
WEB-16	TRACE identifies 35 LLM-judge / human-developer misalignment sources requiring calibration (https://arxiv.org/pdf/2603.24586)
WEB-18	LoCoBench confirms existing benchmarks emphasize completion/comprehension rather than complex software-development workflows (https://arxiv.org/pdf/2509.09614)

Input Ontology (IO)

Score: 1/5 — Serious concern | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Input Ontology definition: Whether the benchmark's test case categories cover the query types expected in deployment.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval's task taxonomy is confined entirely to standalone Python function synthesis assessing 'language comprehension, reasoning, algorithms, and simple mathematics' [\[Q22\]](#). The deployment requires four structurally distinct artifact categories (DB schema, API routes, frontend stubs, IaC) that are wholly absent from the benchmark. The paper itself acknowledges that GitHub code includes 'class implementations, configuration files, scripts, and even files used to store data' that are underrepresented [\[Q48\]](#), and that current synthesis capabilities only address 'tightly specified, constrained problem instances' [\[Q151\]](#). Empirical sampling confirms zero examples of SQL, ORM, REST, frontend, or IaC content across the dataset.

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)
Q151	"The current capabilities of synthesis methodologies are only able to tackle tightly specified, constrained problem instances or narrow tasks." (p.25)

Strengths

- The benchmark does test isolated Python function synthesis with typed signatures, which corresponds to one narrow sub-component (utility helper functions inside FastAPI/SQLAlchemy code) of the broader scaffolding task [\[Q22, DATASET-D2\]](#).

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
D2	<code>`def has_close_elements(numbers: List[float], threshold: float) -> bool: """ Check if in given list of numbers, are any two numbers closer to each other than given threshold."""</code> — Pairwise distance threshold check — pure algorithmic problem

Checklist

Checklist item definitions:

ID	Assessment Question
IO-1	Identify test case categories required for regional deployment
IO-2	Check if taxonomy omits regionally relevant categories
IO-3	Check if taxonomy includes irrelevant categories
IO-4	Document category gaps harming content validity

Checklist responses:

IO-1: Required categories for the deployment: (a) SQL DDL / SQLAlchemy ORM model generation, (b) FastAPI/Spring Boot route handlers with Pydantic/DTO schemas, (c) React/Next.js/Angular component stubs in TypeScript, (d) Dockerfile/Kubernetes/Terraform IaC. None of these are represented in the benchmark's taxonomy.

IO-2: Yes — the benchmark omits all four deployment-required categories. The paper explicitly scopes itself to 'producing functionally correct code bodies from natural language docstrings' [Q130] and characterizes coverage as algorithms/math [Q22]. Empirical sampling found 0/40 examples covering any required category.

IO-3: Yes — competitive-programming puzzle problems (Tribonacci sequences, palindromic 'flying objects', polynomial root finding) consume the entire taxonomy without bearing on enterprise scaffolding. Appendix-D extensions to bias [Q185-Q188] and cryptographic API misuse [Q204-Q205] are partially relevant (security patterns) but still confined to single-function Python.

IO-4: Gaps that harm content validity: complete absence of multi-artifact scaffolding, no framework-idiom categories, no IaC categories, no cross-layer entity-modeling tasks. DevBench [WEB-12] is the nearest available analog and itself reveals LLMs fail repository-level multi-stage tasks.

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)
Q130	"We investigated whether it was possible to train large language models to produce functionally correct code bodies from natural language docstrings." (p.14)
Q151	"The current capabilities of synthesis methodologies are only able to tackle tightly specified, constrained problem instances or narrow tasks." (p.25)
Q185	"In order to better understand the potential that code generation has to encode bias in the context of Codex in particular, we developed a series of probes for instances of harmful bias in single- and multi-line autocompletions." (p.30)
Q186	"We found that, in response to simple prompts like <code>def gender(x):</code> , the generations often assumed binary gender for both single- and multi-line autocompletions." (p.30)
Q187	"When we probed using the prompt <code>def race(x):</code> , we found that many of the most commonly-generated completions assumed a small number of mutually exclusive race categories." (p.30)
Q188	"Most synthesized completions included "White" and many included only a few other categories, followed by "other." Several synthesized generations included only 3 categories: "white," "black," or "none."" (p.30)
Q204	"To study this phenomenon, we asked Codex to suggest code that would call cryptographic libraries to generate cryptographic contexts, and then evaluated whether any of these outputs were clearly insecure." (p.32)
Q205	"When tested on a standard series of prompts asking the models to call functions to produce RSA keys or AES contexts, we find that Codex models of varying sizes frequently use clearly insecure configurations (See Figure 15)." (p.32)
WEB-12	DevBench covers full SDLC stages — illustrates richer output ontology the deployment requires (https://arxiv.org/abs/2403.08604)
WEB-13	FrontendBench covers front-end interactivity output forms (https://arxiv.org/html/2506.13832v2)

Information Gaps

- Whether the 124 unsampled problems contain any framework-idiom content (highly unlikely given paper documentation, but not exhaustively confirmed).

Requires Expert Verification

- Confirmation from the organization's architects that all four artifact layers are required, and that the rubric scope cannot be reduced to a single layer where HumanEval has more applicability.

Remediation

Gap 1: No coverage of DB schema, API route, frontend stub, or IaC artifact categories.

Recommendation: Adopt DevBench [WEB-12] for multi-stage SDLC coverage and add IaC-Eval/DPIaC-Eval/Multi-IaC-Eval [WEB-4/5/6] for the deployment-config layer; build a small org-specific benchmark of PRD→scaffolding examples sampled from the internal architecture guide to cover FastAPI/SQLAlchemy/Pydantic/Spring Boot idioms.

ID	Evidence
WEB-4	IaC-Eval provides Terraform/HCL output-form evaluation HumanEval lacks (https://medium.com/gft-engineering/evaluating-llms-for-infrastructure-as-code-9f8b9ac4ca33)
WEB-12	DevBench covers full SDLC stages — illustrates richer output ontology the deployment requires (https://arxiv.org/abs/2403.08604)

Input Content (IC)

Score: 2/5 — Significant gaps | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Input Content definition: Whether individual datapoint content is culturally and contextually appropriate for the target region.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

Content is hand-written algorithmic/interview-style problems [\[Q14, Q19, Q131\]](#) sourced from no enterprise-development context. Training data is GitHub Python at large [\[Q23, Q25\]](#), and the Codex-S fine-tune adds competitive-programming and CI-traced functions [\[Q52, Q55\]](#) — none of which encode FastAPI/SQLAlchemy/Pydantic/Spring Boot idioms, business data models, compliance constraints, or SLA requirements. The natural language *register* is US English [\[DATASET-D7\]](#) which matches the deployment, and typed Python signatures appear [\[DATASET-D2, DATASET-D6\]](#) — narrow points of contact. The security probe content [\[Q206\]](#) is operationally grounded but still single-function. No content addresses the deployment's PRD-embedded business context, non-functional requirements, or convention encoding.

ID	Evidence
Q14	"Though not a guarantee for problem novelty, all problems were hand-written and not programmatically copied from existing sources." (p.3)
Q19	"We evaluate functional correctness on a set of 164 hand-written programming problems, which we call the HumanEval dataset." (p.4)
Q23	"Our training dataset was collected in May 2020 from 54 million public software repositories hosted on GitHub, containing 179 GB of unique Python files under 1 MB." (p.4)
Q25	"After filtering, our final dataset totaled 159 GB." (p.4)
Q52	"We collected problem statements, function signatures, and solutions from several popular programming contest and interview preparation websites. We then assembled these into programming tasks similar to HumanEval, using the problem description as the docstring. Since complete test suites are often hidden, we created unit tests from examples found in the problem statements, or extracted additional test cases through submitting incorrect solutions. In total, we curated 10,000 problems in this way." (p.7)
Q55	"We considered GitHub repos using travis and tox as their CI frameworks, as they are two of the most popular CI tools. We additionally used publicly available source code from pip packages found in the python package index (PyPI)." (p.7)
Q131	"By fine-tuning GPT on code from GitHub, we found that our models displayed strong performance on a dataset of human-written problems with difficulty level comparable to easy interview problems." (p.14)
Q206	"We used 5 prompts across different libraries for RSA and AES based on Sonar Source's Python vulnerability database, and generated ~30k samples total." (p.32)
D2	<code>`def has_close_elements(numbers: List[float], threshold: float) -> bool: """ Check if in given list of numbers, are any two numbers closer to each other than given threshold. """</code> — Pairwise distance threshold check — pure algorithmic problem
D6	<code>`def parse_music(music_string: str) -> List[int]: """ Input to this function is a string representing musical notes in a special ASCII format. Your task is to parse this string and return list of integers corresponding to how many beats does each not last. """</code> — ASCII music note parser — short docstring, single function
D7	<code>`def valid_date(date): """You have to write a function which validates a given date string and returns True if the date is valid otherwise False. The date should be in the format: mm-dd-yyyy"""</code> — Date validation — single utility function, no framework idioms

Strengths

- All docstrings are in standard US English, matching the deployment's exclusive working language [\[DATASET-D7\]](#).
- Some problems use modern typed Python signatures (List[float], etc.) consistent with Pydantic/FastAPI typing expectations [\[DATASET-D2, DATASET-D6\]](#).

- Security probe content [Q206] adds a small amount of operational vulnerability-pattern content relevant to enterprise security review.

ID	Evidence
Q206	"We used 5 prompts across different libraries for RSA and AES based on Sonar Source's Python vulnerability database, and generated ~30k samples total." (p.32)
D2	`def has_close_elements(numbers: List[float], threshold: float) -> bool: """ Check if in given list of numbers, are any two numbers closer to each other than given threshold. """` — Pairwise distance threshold check — pure algorithmic problem
D6	`def parse_music(music_string: str) -> List[int]: """ Input to this function is a string representing musical notes in a special ASCII format. Your task is to parse this string and return list of integers corresponding to how many beats does each note last. """` — ASCII music note parser — short docstring, single function
D7	`def valid_date(date): """ You have to write a function which validates a given date string and returns True if the date is valid otherwise False. The date should be in the format: mm-dd-yyyy """` — Date validation — single utility function, no framework idioms

Checklist

Checklist item definitions:

ID	Assessment Question
IC-1	Do inputs require region-specific cultural/geographic/dialectal knowledge?
IC-2	Does culturally sensitive content align with target culture?
IC-3	Flag inputs assuming Western-specific knowledge
IC-4	Regional annotator recruitment for culturally sensitive instances
IC-5	Document content issues harming content validity

Checklist responses:

IC-1: The deployment requires enterprise-software cultural knowledge (FastAPI conventions, Spring Boot annotations, Kubernetes manifest structure, AWS IaC patterns), not regional/dialectal cultural knowledge in the traditional sense. The benchmark's content does not encode any of this enterprise-stack knowledge [Q22, DATASET-D4].

IC-2: The puzzle/whimsical register ('will it fly', musical-note parsing, fruit-distribution word problems) is categorically different from enterprise-development prose [DATASET-D22, DATASET-D6, DATASET-D21]. Geographically the benchmark is US/Anglophone-origin which matches the target population.

IC-3: Western-specific knowledge is not the main issue; the issue is competitive-programming-specific framing. One observed locale artifact (comma-as-decimal in HumanEval/137 [DATASET-D11]) is European in origin but irrelevant to the deployment.

IC-4: INSUFFICIENT DOCUMENTATION — annotator demographics are not provided; the paper states 'there is unfortunately only limited research on the demographic distribution of Python users' [Q219]. Re-annotation would require org-internal architects (already available as the grading population).

IC-5: Content harms validity by failing to probe framework idioms, business entity modeling, or compliance/SLA constraints; training-data quality issues are acknowledged [Q167] but do not bear directly on the deployment task.

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
Q167	"We believe this is unlikely to be a large factor here, as the GitHub dataset contains plenty of poor-quality code. The bugs are designed to be of the sort we'd expect to appear commonly in the dataset; code that compiles and often runs without errors but gives an incorrect answer. Examples include off-by-one errors or single-character typographic errors." (p.27)
Q219	"There is unfortunately only limited research on the demographic distribution of Python users." (p.33)
D4	`def find_zero(xs: list): """ xs are coefficients of a polynomial. find_zero find x such that poly(x) = 0. """` — Numerical root-finding for polynomial — algorithm/math problem

D6	<code>`def parse_music(music_string: str) -> List[int]: """ Input to this function is a string representing musical notes in a special ASCII format. Your task is to parse this string and return list of integers corresponding to how many beats does each not last."""`</code> — ASCII music note parser — short docstring, single function
D11	<code>`def compare_one(a, b): """ Create a function that takes integers, floats, or strings representing real numbers, and returns the larger variable in its given variable type. Note: If a real number is represented as a string, the floating point might be . or , """`</code> — Numeric comparison with comma-decimal format — minor internationalisation note
D21	<code>`def fruit_distribution(s,n): """ In this task, you will be given a string that represents a number of apples and oranges that are distributed in a basket of fruit this basket contains apples, oranges, and mango fruits."""`</code> — Word-problem arithmetic parsing — no enterprise relevance
D22	<code>`def will_it_fly(q,w): """ Write a function that returns True if the object q will fly, and False otherwise. The object q will fly if it's balanced (it is a palindromic list) and the sum of its elements is less than or equal the maximum possible weight w."""`</code> — Palindrome + sum constraint — whimsical puzzle framing
WEB-22	Faros.ai — 153% increase in architectural flaws in AI-generated code; 322% increase in privilege escalation paths (https://www.faros.ai/blog/enterprise-ai-coding-assistant-adoption-scaling-guide)

Information Gaps

- Annotator demographics and selection process for the 164 problems are not documented.
- Whether any framework-specific patterns appear in the Codex-S 40k CI-traced functions (training-side, not in HumanEval test set).

Requires Expert Verification

- Architect-team review of whether the security probe content [Q206] has any reusable value for evaluating auth-middleware or secrets-handling correctness in the deployment.

Remediation

Gap 1: Absence of framework-idiomatic content, business entity models, and NFR/compliance constraints.

Recommendation: Construct an internal evaluation set from anonymized real PRDs paired with architect-approved scaffolding solutions; capture the full content register (business context, entity definitions, NFRs, auth/SLA constraints) that HumanEval omits.

Input Form (IF)

Score: 1/5 — Serious concern | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Input Form definition: Whether the input signal encoding (text, audio, image parameters) matches deployment conditions.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval inputs are short single-paragraph docstrings averaging 7.7 tests per problem **[Q20]**, assembled as 'a header, a signature, and a docstring' **[Q28]**. The deployment requires 3–10 page semi-structured PRDs with consistent headings and variable prose under each section. The paper itself explicitly acknowledges Codex 'struggles to parse through increasingly long and higher-level or system-level specifications' **[Q88]** and demonstrates exponential degradation with docstring complexity **[Q90, Q93]**. Empirical sampling confirms docstrings range from a single sentence **[DATASET-D13, DATASET-D14]** to ~15 lines maximum **[DATASET-D7]** — orders of magnitude shorter than a 3–10 page PRD. LongCodeBench **[WEB-3]** documents Claude 3.5 Sonnet dropping from 29% to 3% as context length increases on coding tasks, quantifying the severity of this mismatch.

ID	Evidence
Q20	"Each problem includes a function signature, docstring, body, and several unit tests, with an average of 7.7 tests per problem." (p.4)
Q28	"To compute pass@k, we assemble each HumanEval problem into a prompt consisting of a header, a signature, and a docstring, which is illustrated in Figure 2." (p.4)
Q88	"Moreover, Codex struggles to parse through increasingly long and higher-level or system-level specifications." (p.10)
Q90	"We find that as the number of chained building blocks in the docstring increases, model performance decreases exponentially." (p.10)
Q93	"With each additional component, pass rate drops by roughly a factor of 2-3." (p.10)
D7	<code>`def valid_date(date): """You have to write a function which validates a given date string and returns True if the date is valid otherwise False. The date should be in the format: mm-dd-yyyy"""`</code> — Date validation — single utility function, no framework idioms
D13	<code>`def incr_list(l: list): """Return list with elements incremented by 1."""`</code> — Trivially simple list transform — minimal docstring
D14	<code>`def below_threshold(l: list, t: int): """Return True if all numbers in the list l are below threshold t."""`</code> — Threshold comparison — very short docstring
WEB-3	LongCodeBench documents Claude 3.5 Sonnet dropping from 29% to 3% as context length increases on coding tasks (https://arxiv.org/pdf/2505.07897)

Strengths

- Input form within HumanEval is clean and parseable, making the benchmark easy to load and run as a lower-bound check for single-function Python utilities embedded within the deployment's broader output **[DATASET-D1]**.

ID	Evidence
D1	<code>`def correct_bracketing(brackets: str): """ brackets is a string of "(" and ")". return True if every opening bracket has a corresponding closing bracket."""`</code> — Classic bracket-matching algorithm problem — no enterprise context

Checklist

Checklist item definitions:

ID	Assessment Question
----	---------------------

IF-1	Compare signal distributions between source and target
IF-2	Check if regional infrastructure supports data capture specs
IF-3	Identify domain-specific form differences
IF-4	Document form mismatches harming external validity

Checklist responses:

IF-1: Signal-distribution mismatch is severe: short docstrings (~1 sentence to ~15 lines) vs. 3–10 page semi-structured PRDs with multi-section headings. The paper acknowledges this kind of long-specification handling is a known weakness [Q88].

IF-2: Infrastructure question is largely inapplicable — both contexts use text. However, stop-sequence tuning ('\nclass', '\ndef', '\n#', '\nif', '\nprint') [Q29] is configured for single-function termination and would need redesign for multi-file output.

IF-3: Domain-specific form differences: HumanEval lacks structured-document parsing (headings, user-story templates, NFR tables); requires no long-context attention; provides no multi-section coherence test. LongCodeBench [WEB-3] is the closest published evidence base on long-context degradation. LoCoBench [WEB-18] explicitly notes existing benchmarks 'primarily emphasize code completion and comprehension tasks rather than complex software development workflows'.

IF-4: External validity is severely harmed: short-docstring pass rates do not predict PRD-length input performance; quantified degradation rate (29%→3% [WEB-3]) provides empirical justification. The benchmark cannot surface context-length failure modes at all.

ID	Evidence
Q20	"Each problem includes a function signature, docstring, body, and several unit tests, with an average of 7.7 tests per problem." (p.4)
Q28	"To compute pass@k, we assemble each HumanEval problem into a prompt consisting of a header, a signature, and a docstring, which is illustrated in Figure 2." (p.4)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\nclass', '\ndef', '\n#', '\nif', or '\nprint', since the model will continue generating additional functions or statements otherwise." (p.4)
Q88	"Moreover, Codex struggles to parse through increasingly long and higher-level or system-level specifications." (p.10)
Q90	"We find that as the number of chained building blocks in the docstring increases, model performance decreases exponentially." (p.10)
Q93	"With each additional component, pass rate drops by roughly a factor of 2-3." (p.10)
WEB-3	LongCodeBench documents Claude 3.5 Sonnet dropping from 29% to 3% as context length increases on coding tasks (https://arxiv.org/pdf/2505.07897)
WEB-18	LoCoBench confirms existing benchmarks emphasize completion/comprehension rather than complex software-development workflows (https://arxiv.org/pdf/2509.09614)

Information Gaps

- Exact token-count distribution of the org's PRD corpus is not measured (noted as [NEEDS VERIFICATION] in regional context).

Requires Expert Verification

- Architect-team measurement of typical PRD token counts on the org's actual document corpus to size context-window requirements precisely.

Remediation

Gap 1: Short docstrings cannot probe long-context PRD parsing (29%→3% Claude 3.5 Sonnet degradation [WEB-3]).

Recommendation: Add LongCodeBench [WEB-3] / LoCoBench [WEB-18] to the evaluation suite and measure typical PRD token counts on the org's actual document corpus to calibrate context-window sizing; compare full-document vs. section-by-section prompting empirically.

ID	Evidence
WEB-3	LongCodeBench documents Claude 3.5 Sonnet dropping from 29% to 3% as context length increases on coding tasks (https://arxiv.org/pdf/2505.07897)
WEB-18	LoCoBench confirms existing benchmarks emphasize completion/comprehension rather than complex software-development workflows (https://arxiv.org/pdf/2509.09614)

Output Ontology (OO)

Score: 1/5 — Serious concern | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Output Ontology definition: Whether the benchmark's output categories and scoring criteria reflect regionally valid decision boundaries.

Each section below is followed by an evidence table. Three types of evidence are cited: [QN] — verbatim quotes extracted from the benchmark paper; [WEB-N] — web sources consulted during assessment; [DN] / [DATASET-DN] — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval's output ontology is binary functional correctness via unit tests [Q77]. The paper argues this is superior to BLEU-style match metrics [Q10, Q18, Q40] — a defensible position for algorithm problems — but explicitly acknowledges that 'evaluating code generation' for tightly specified tasks is different from broader qualitative metrics it can only prospectively describe [Q86, Q149], and that 'there is no similar way to evaluate docstring samples automatically' [Q78]. The deployment requires a four-layer rubric (syntactic/lint, requirements completeness, architectural convention adherence, multi-file coherence) — none of which can be expressed by pass/fail unit tests. Appendix-D specification attributes (variable interdependencies [Q153], temporal reasoning [Q154], concurrency [Q155]) and the security-probe ontology (RSA <2048 bits [Q208], AES ECB [Q209]) extend the schema slightly but remain incompatible with rubric-based architectural grading. Prometheus 2 [WEB-14] offers a candidate rubric-judge framework that the org would need to deploy separately.

ID	Evidence
Q10	"More fundamentally, match-based metrics are unable to account for the large and complex space of programs functionally equivalent to a reference solution." (p.2)
Q18	"Later, we provide evidence that BLEU score may not be a reliable indicator of functional correctness by showing that functionally inequivalent programs generated by our model (which are guaranteed to disagree with the reference solution on some input) often have higher BLEU scores than functionally equivalent ones." (p.3)
Q40	"Finally, we compute BLEU scores for all Codex-12B HumanEval samples (at temperature 0.8) against their reference solutions. For each problem, when we plot the distributions of BLEU scores for correct and incorrect solutions, we notice significant overlap (Figure 8). Since an incorrect solution is guaranteed to be functionally inequivalent to the reference solution, we conclude that improvements in BLEU score may not indicate improved rates of functional correctness in practice." (p.6)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q78	"However, there is no similar way to evaluate docstring samples automatically. Therefore, we grade sample docstrings by hand, considering a docstring correct if it uniquely and accurately specifies the code body." (p.9)
Q86	"While evaluating code generation is well-studied (Xu et al., 2021; Helmuth & Spector, 2015; Pantridge et al., 2017), many existing metrics measure performance in tightly specified, constrained problem instances (e.g., string manipulation in FlashFill (Gulwani, 2011)). Therefore, we developed a set of qualitative metrics for measuring the capabilities of code generating models while controlling for the complexity and abstraction level of the specifications (Appendix D)." (p.10)
Q149	"Below we provide brief descriptions of such attributes and qualitative metrics, which are to be further discussed in a forthcoming paper along with associated results for Codex models." (p.25)
Q153	"Variable Interdependencies: Tracking state of more than one variable, their interdependencies and nesting, all possible permutations of state, and the relationship between input and output parameters" (p.25)
Q154	"Temporal Reasoning: as consideration of future and past program states including Safety properties entailing that a defined "bad" state never occurs and Liveness properties entailing progress towards a specific goal or state" (p.25)
Q155	"Concurrency and Parallelism: Correct and sound reasoning over computational interleavings (for various specification granularities)." (p.25)
Q208	"RSA keys were considered improperly configured if they were shorter than 2048 bits." (p.32)
Q209	"AES contexts were considered improperly configured if they used the ECB cipher mode (see Menezes et al. (2018), p. 228)." (p.32)
WEB-14	Prometheus 2 supports user-defined rubric criteria in pointwise and pairwise formats (https://arxiv.org/abs/2405.01535)

Strengths

- Pass/fail unit-test scoring provides a usable lower-bound signal for the 'syntactic/lint correctness' layer of the deployment rubric where Python utility functions are concerned [Q77, DATASET-D1].
- The security probe ontology (RSA key length, AES mode) [Q208, Q209] provides a small precedent for adding programmatic correctness checks beyond unit tests, partially reusable for the deployment's security-pattern checks.

ID	Evidence
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q208	"RSA keys were considered improperly configured if they were shorter than 2048 bits." (p.32)
Q209	"AES contexts were considered improperly configured if they used the ECB cipher mode (see Menezes et al. (2018), p. 228)." (p.32)
D1	<code>`def correct_bracketing(brackets: str): """ brackets is a string of "(" and ")". return True if every opening bracket has a corresponding closing bracket."""`</code> — Classic bracket-matching algorithm problem — no enterprise context

Checklist

Checklist item definitions:

ID	Assessment Question
OO-1	Review output label categories for regional relevance
OO-2	Identify missing regionally specific categories
OO-3	Flag categories encoding non-regional values
OO-4	Consider stakeholder-driven taxonomy redesign
OO-5	Document taxonomy issues harming structural/content validity

Checklist responses:

OO-1: The deployment's label space (4-layer rubric with multiple valid designs) is fundamentally categorical/multi-dimensional. HumanEval's binary unit-test pass/fail label has minimal overlap.

OO-2: Missing categories: syntactic/lint compliance scoring (not just execution), requirements-completeness coverage, architectural convention adherence (naming, folder structure, auth patterns), multi-file coherence (entity ↔ API ↔ frontend type alignment). Cross-layer consistency is unmeasurable [WEB-11, WEB-12].

OO-3: The benchmark's ontology assumes a single correct behavioral contract per problem, which conflicts with the deployment's explicit acceptance of multiple valid architectural designs (correctness_model.multiple_valid_designs=true).

OO-4: Stakeholder-driven redesign is required: the architecture guide must be encoded as a structured rubric (Prometheus 2-compatible [WEB-14]) and validated against human gold-annotated samples, with TRACE [WEB-16] biases addressed (LLM-judge preference for longer code, etc.).

OO-5: Structural validity is harmed because the construct's structure (multi-dimensional architectural correctness) is collapsed to a single binary axis. Content validity is harmed by missing categories. External validity is harmed because HumanEval scores cannot predict rubric-based performance. IaC performance floors of <20% pass@1 [WEB-4] and ~50% on simplest scenarios [WEB-5] confirm that ontology gaps in HumanEval map to known LLM weakness areas.

ID	Evidence
Q10	"More fundamentally, match-based metrics are unable to account for the large and complex space of programs functionally equivalent to a reference solution." (p.2)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)

Q78	"However, there is no similar way to evaluate docstring samples automatically. Therefore, we grade sample docstrings by hand, considering a docstring correct if it uniquely and accurately specifies the code body." (p.9)
Q86	"While evaluating code generation is well-studied (Xu et al., 2021; Helmuth & Spector, 2015; Pantridge et al., 2017), many existing metrics measure performance in tightly specified, constrained problem instances (e.g., string manipulation in FlashFill (Gulwani, 2011)). Therefore, we developed a set of qualitative metrics for measuring the capabilities of code generating models while controlling for the complexity and abstraction level of the specifications (Appendix D)." (p.10)
Q149	"Below we provide brief descriptions of such attributes and qualitative metrics, which are to be further discussed in a forthcoming paper along with associated results for Codex models." (p.25)
WEB-4	laC-Eval provides Terraform/HCL output-form evaluation HumanEval lacks (https://medium.com/gft-engineering/evaluating-llms-for-infrastructure-as-code-9f8b9ac4ca33)
WEB-5	DPIaC-Eval finds 42.7% of syntactically correct templates fail deployment, demonstrating ontology layers HumanEval cannot capture (https://arxiv.org/html/2506.05623v1)
WEB-11	DevEval introduces Recall@k for dependency accuracy — closest published cross-file coherence metric (https://arxiv.org/pdf/2405.19856)
WEB-12	DevBench covers full SDLC stages — illustrates richer output ontology the deployment requires (https://arxiv.org/abs/2403.08604)
WEB-14	Prometheus 2 supports user-defined rubric criteria in pointwise and pairwise formats (https://arxiv.org/abs/2405.01535)
WEB-16	TRACE identifies 35 LLM-judge / human-developer misalignment sources requiring calibration (https://arxiv.org/pdf/2603.24586)

Information Gaps

- Whether the architecture guide can be reduced to a rubric format compatible with Prometheus 2-style judging without losing fidelity.

Requires Expert Verification

- Architect-team encoding of the internal architecture guide into a structured criterion-separated rubric.
- Calibration of any rubric-judge model against human-annotated gold samples per TRACE recommendations [WEB-16].

Remediation

Gap 1: Binary unit-test pass/fail cannot express the deployment's 4-layer rubric (syntactic/lint, requirements completeness, architectural convention adherence, multi-file coherence).

Recommendation: Encode the internal architecture guide as a structured Prometheus 2-compatible rubric [WEB-14], with criterion-separated scoring; calibrate the judge model against a small human-annotated gold set per TRACE recommendations [WEB-16]; complement with deterministic checks (linters, schema diffing) where possible.

ID	Evidence
WEB-14	Prometheus 2 supports user-defined rubric criteria in pointwise and pairwise formats (https://arxiv.org/abs/2405.01535)
WEB-16	TRACE identifies 35 LLM-judge / human-developer misalignment sources requiring calibration (https://arxiv.org/pdf/2603.24586)

Output Content (OC)

Score: 1/5 — Serious concern | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Output Content definition: Whether ground-truth labels align with the judgments of regional stakeholders.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

Ground-truth label provenance for HumanEval is sparse and inapplicable to the deployment. The paper provides no annotator demographics, no inter-annotator agreement, and only minimal documentation of who hand-wrote the problems. The only explicit human annotation activity is hand-grading 10 docstring samples per problem 'due to the time consuming nature of this process' [\[Q79\]](#). Unit-test labels for the Codex-S competitive-programming subset were partially constructed from problem-statement examples rather than complete hidden suites [\[Q52\]](#), and the paper acknowledges 'human developers often write test suites with limited but targeted coverage' [\[Q129\]](#). Alignment evaluations treat the model itself as the labeler [\[Q164\]](#). For a deployment whose ground truth requires expert architectural judgment against an internal style guide written by senior architects, HumanEval's label provenance is entirely inapplicable. Empirical sampling further confirms vacuous filler assertions [\[DATASET-D17\]](#) and minor annotation quality issues [\[DATASET-D27\]](#) suggesting no formal multi-reviewer process.

ID	Evidence
Q52	"We collected problem statements, function signatures, and solutions from several popular programming contest and interview preparation websites. We then assembled these into programming tasks similar to HumanEval, using the problem description as the docstring. Since complete test suites are often hidden, we created unit tests from examples found in the problem statements, or extracted additional test cases through submitting incorrect solutions. In total, we curated 10,000 problems in this way." (p.7)
Q79	"Due to the time consuming nature of this process, we only grade 10 samples per problem, for a total of 1640 problems, from Codex-D-12B at temperature 0.8." (p.9)
Q129	"Human developers often write test suites with limited but targeted coverage, but this does not always work well against an algorithm, highlighting the challenges of evaluating correctness of programs." (p.14)
Q164	"We instruct the model to write correct code, and we assume the model could easily be fine-tuned to detect such an instruction. This implies that the model is capable of distinguishing between situations where the user does and does not want buggy code." (p.27)
D17	<code>`def by_length(arr): """ Given an array of integers, sort the integers that are between 1 and 9 inclusive, reverse the resulting array, and then replace each digit by its corresponding name from "One", "Two", "Three"..."""</code> — Word-for-digit mapping — contrived puzzle task
D27	<code>`def digitSum(s): """Task Write a function that takes a string as input and returns the sum of the upper characters only' ASCII codes.""</code> — ASCII uppercase character sum — simple string processing

Strengths

- Unit-test labels in the sampled set generally cover edge cases (empty inputs, format violations, boundary conditions) [\[DATASET-D1, DATASET-D7\]](#), so where HumanEval is reused as a sub-component check for Python utility functions, the labels provide meaningful behavioral signal.

ID	Evidence
D1	<code>`def correct_bracketing(brackets: str): """ brackets is a string of "(" and ")"". return True if every opening bracket has a corresponding closing bracket.""</code> — Classic bracket-matching algorithm problem — no enterprise context
D7	<code>`def valid_date(date): """You have to write a function which validates a given date string and returns True if the date is valid otherwise False. The date should be in the format: mm-dd-yyyy""</code> — Date validation — single utility function, no framework idioms

Checklist

Checklist item definitions:

ID	Assessment Question
OC-1	Do ground-truth labels reflect regional stakeholder perspectives?
OC-2	Assess potential annotator-regional population disagreement
OC-3	Review annotator demographics from documentation
OC-4	Consider label re-annotation by regional annotators
OC-5	Review aggregation methods for minority perspective erasure
OC-6	Document label issues harming convergent/external validity

Checklist responses:

OC-1: No — labels reflect algorithmic-correctness ground truth, not architectural-convention ground truth required by US enterprise architects [Q77, DATASET-D3].

OC-2: Potential disagreement is total: HumanEval ground truth has no architectural-convention dimension to disagree about. A model passing all unit tests may still violate every internal convention. Faros.ai analysis [WEB-22] documents 153% increase in architectural flaws in AI-generated code — precisely the failure mode HumanEval cannot label.

OC-3: INSUFFICIENT DOCUMENTATION — the paper provides no Datasheet, no Data Statement, no annotator demographics, no professional-background information, no inter-annotator agreement statistics. The paper itself notes limited research on Python-user demographics [Q219].

OC-4: Re-annotation would require the org's own architects working from the internal architecture guide — effectively building a new benchmark, not relabeling HumanEval.

OC-5: INSUFFICIENT DOCUMENTATION — no aggregation method is documented because labels are binary per-task and not aggregated across annotators.

OC-6: Convergent validity and external validity are both violated: original labels do not correlate with regional/organizational stakeholder judgments because they probe a different construct entirely. Stack Overflow 2025 [WEB-1] also notes 76% of developers resist AI for 'deployment and monitoring' tasks — the very tasks HumanEval's labels cannot evaluate.

ID	Evidence
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q129	"Human developers often write test suites with limited but targeted coverage, but this does not always work well against an algorithm, highlighting the challenges of evaluating correctness of programs." (p.14)
Q164	"We instruct the model to write correct code, and we assume the model could easily be fine-tuned to detect such an instruction. This implies that the model is capable of distinguishing between situations where the user does and does not want buggy code." (p.27)
Q219	"There is unfortunately only limited research on the demographic distribution of Python users." (p.33)
D3	<code>`def tri(n): """Everyone knows Fibonacci sequence... Tribonacci sequence is defined by the recurrence: tri(1) = 3, tri(n) = 1 + n / 2, if n is even."""`</code> — Mathematical sequence computation — competitive-programming style
WEB-1	Stack Overflow 2025 — 76% of developers don't plan to use AI for deployment/monitoring; 69% don't plan to use AI for project planning — reflects target-cohort skepticism about LLM outputs in exactly the layers HumanEval cannot grade (https://survey.stackoverflow.co/2025/ai)
WEB-22	Faros.ai — 153% increase in architectural flaws in AI-generated code; 322% increase in privilege escalation paths (https://www.faros.ai/blog/enterprise-ai-coding-assistant-adoption-scaling-guide)

Information Gaps

- Who hand-wrote the 164 problems and what calibration process was applied is not documented.
- Whether multiple reviewers assessed each problem is not documented.

- Inter-annotator agreement for the docstring grading task is not reported.

Requires Expert Verification

- Construction of a new gold-annotated sample set by the org's architects against the internal architecture guide if HumanEval-style labels are to be supplemented for the deployment.
-

Remediation

Gap 1: No annotator demographics, no inter-annotator agreement, no architectural-convention labels.

Recommendation: Recruit 2–3 senior architects from the target team to independently grade a sample of model outputs against the architecture guide; measure inter-annotator agreement to validate the rubric and identify ambiguous criteria before scaling.

Output Form (OF)

Score: 2/5 — Significant gaps | Confidence: high

Benchmark: HUMANEVAL | Context: US Enterprise Software Architects — Full-Stack Scaffolding Generation

Output Form definition: Whether the expected output modality matches regional deployment needs and accessibility.

Each section below is followed by an evidence table. Three types of evidence are cited: [QN] — verbatim quotes extracted from the benchmark paper; [WEB-N] — web sources consulted during assessment; [DN] / [DATASET-DN] — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval outputs are single Python function completions [Q2] evaluated by deterministic unit-test execution with nucleus sampling top_p=0.95 [Q30] and temperature optimized per k [Q34]. The deployment requires multi-file project trees spanning Python (FastAPI/SQLAlchemy/Pydantic), SQL, TypeScript (React/Angular), Java (Spring Boot), YAML (Docker/Kubernetes), and HCL (Terraform). Stop sequences are explicitly configured for single-function termination ('\nclass', '\ndef', etc.) [Q29], which would require complete redesign for multi-file output. The mean-log-probability sample-selection heuristic [Q39] is structurally closer to the deployment's single-completion API use case than to pass@k, so partial reusability exists for the autocomplete-style aspect. Empirical sampling confirms zero multi-file or non-Python outputs across the dataset.

ID	Evidence
Q2	"On HumanEval, a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%." (p.1)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\nclass', '\ndef', '\n#', '\nif', or '\nprint', since the model will continue generating additional functions or statements otherwise." (p.4)
Q30	"We use nucleus sampling (Holtzman et al., 2020) with top p = 0.95 for all sampling evaluation in this work." (p.4)
Q34	"In particular, for a 679M parameter model, the optimal temperature for pass@1 is T* = 0.2 and the optimal temperature for pass@100 is T* = 0.8." (p.5)
Q39	"Inspired by similar work in language modeling, we find that choosing the sample with the highest mean token log probability outperforms evaluating a random sample, while choosing the sample based on sum log probability can perform slightly worse than picking randomly." (p.5)

Strengths

- The text-in / code-out modality matches the deployment at the signal level [Q2].
- Single-completion sample selection via mean log probability [Q39] is structurally aligned with the deployment's single-prompt LLM-API usage pattern.
- Sandboxed execution infrastructure [Q8, DATASET-D1] is directly reusable for evaluating any Python sub-components in the deployment's output.

ID	Evidence
Q2	"On HumanEval, a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%." (p.1)
Q8	"In this section, we discuss the details of our evaluation framework. We begin by defining the pass@k metric, and explain its advantages over standard match-based metrics. Next, we describe the dataset of hand-written problems, called "HumanEval," which we created in order to benchmark our models. Finally, we discuss the sandbox environment we used to safely execute model-generated code." (p.2)
Q39	"Inspired by similar work in language modeling, we find that choosing the sample with the highest mean token log probability outperforms evaluating a random sample, while choosing the sample based on sum log probability can perform slightly worse than picking randomly." (p.5)
D1	`def correct_bracketing(brackets: str): """ brackets is a string of "(" and ")". return True if every opening bracket has a corresponding closing bracket."""` — Classic bracket-matching algorithm problem — no enterprise context

Checklist

Checklist item definitions:

ID	Assessment Question
OF-1	Does output modality match regional deployment needs?
OF-2	Assess text-to-speech availability for speech output
OF-3	Consider literacy rates and accessibility requirements
OF-4	Document form mismatches harming external validity

Checklist responses:

OF-1: Partial match — both contexts output text-form code, but HumanEval is restricted to single-file single-function Python while the deployment requires multi-file, multi-language output trees [Q2 vs. [deployment.artifact_layers_under_evaluation](#)].

OF-2: Not applicable — neither context involves speech output.

OF-3: Not applicable — target population is professional engineers with expert technical literacy [regional context: [technical_literacy](#)].

OF-4: External validity is moderately harmed: HumanEval cannot evaluate multi-file outputs, cross-language outputs (SQL, TypeScript, YAML, HCL), or structural properties (folder layout, import graph coherence). DevEval Recall@k [WEB-11] is the closest published metric for cross-file dependency accuracy. IaC-Eval [WEB-4] and Multi-IaC-Eval [WEB-6] cover the YAML/HCL output form HumanEval omits entirely.

ID	Evidence
Q2	"On HumanEval, a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%." (p.1)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\nclass', '\ndef', '\n#', '\nif', or '\nprint', since the model will continue generating additional functions or statements otherwise." (p.4)
WEB-4	IaC-Eval provides Terraform/HCL output-form evaluation HumanEval lacks (https://medium.com/gft-engineering/evaluating-llms-for-infrastructure-as-code-9f8b9ac4ca33)
WEB-6	Multi-IaC-Eval covers CloudFormation, Terraform, and CDK output forms (https://arxiv.org/pdf/2509.05303)
WEB-11	DevEval introduces Recall@k for dependency accuracy — closest published cross-file coherence metric (https://arxiv.org/pdf/2405.19856)

Information Gaps

- Whether the org would treat partial-output reuse (e.g., extracting the Python-only sub-component score) as meaningful signal for any deployment decision.

Requires Expert Verification

- Architect-team decision on whether multi-file coherence metrics (e.g., entity-name diffing across generated artifact layers) should be implemented as custom evaluation harnesses to supplement HumanEval-style execution checks.

Remediation

Gap 1: Single-file single-function output cannot evaluate multi-file project trees or cross-language coherence.

Recommendation: Build an in-house harness for multi-file coherence checks (entity-name diffing between DB schema, API model, frontend prop types; import-graph validation) since no public benchmark explicitly scores this; reuse DevEval's Recall@k [WEB-11] as a starting metric for cross-file dependency accuracy.

ID	Evidence
----	----------

WEB-11

DevEval introduces Recall@k for dependency accuracy — closest published cross-file coherence metric (<https://arxiv.org/pdf/2405.19856>)